The CPU: A very tall pile of simple

Part 1: Foundations

Ekansh Goenka

October 2026

Contents

1.1. How to Read This 4
1.2. Circuits & Electricity 5
1.3. Switches, Relays, & Logic Gates 5
1.4. Counting With Wires 15
1.5. Addition 19

You can hear the phrase

"computers think in 1s and 0s"

a hundred times and still not understand how a computer actually works. By itself, this explains basically nothing. Sure, a wire can be high or low. Sure, a light can be on or off. But how does that become addition?

How does that become memory?

How does that become a program sitting in `RAM`, one instruction after another, telling a machine what to do?

Some resources stay extremely high-level, so you never really understand how a CPU actually works.

The deeper resources are amazing, but they are long, dense, and intimidating. And frankly, for someone who doesn't want that level of detail, a lot of it can often feel like too much.

My goal is to help you understand what is going on under the hood, without exploding your brain or eating weeks of time.

We start with a simple circuit turning a light bulb on and off, then work our way through logic gates, memory, and the basic circuits underneath them.

The key point is that nothing here is smart in isolation. A CPU is not one hard idea. It is a very tall pile of simple ones.

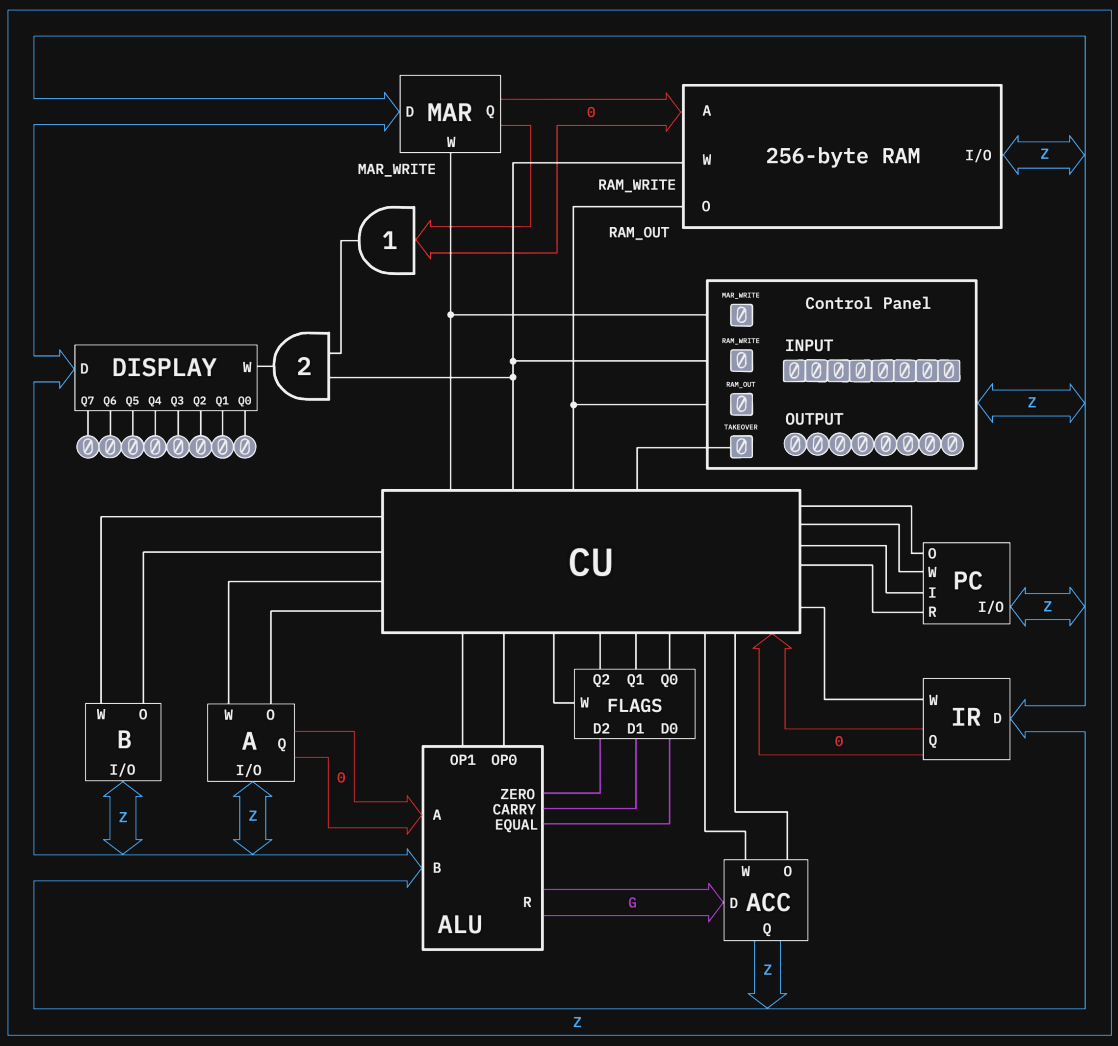


Diagram 1.1. The whole CPU.

This probably looks like an overwhelming mess, but don't worry, that's normal. We are going to try to understand this simple CPU. It is not a modern CPU with decades of optimization, but it has the same core functionality.

1.1. How to Read This

- If a diagram is hard to understand, click it and step through each frame one by one using the arrow keys, with the provided descriptions. This only works if you are reading on my website.
- Don't try to "memorize" every layout. Focus on the mental models, concepts, and what part each piece has to play.
- If a section feels dense or hard to understand, follow the diagrams first and try to get a feel for what is happening.

1.2. Circuits & Electricity

First, we need the basics of how electricity and circuits work.

Here is a simple circuit:

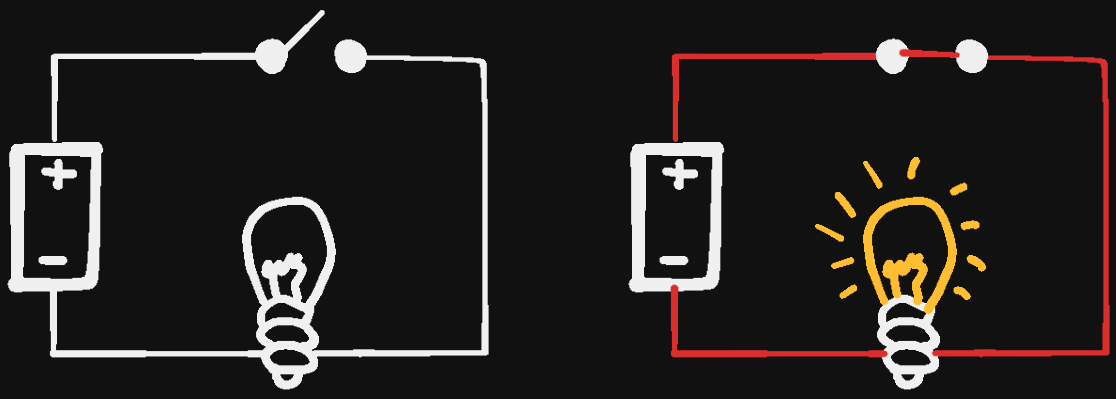


Diagram 3.1. A basic circuit with a battery, switch, and bulb.

We can think of the battery as being able to push charge around the loop. Current can only flow when this loop is completed.

If the loop is broken, nothing flows. A switch is simply a controlled break in the loop, allowing us to break and complete the loop whenever we want.

And a light bulb is just a simple light bulb. It glows when current flows through the filament.

Now we have a circuit that can do one yes/no thing. Current flows or it doesn't.

Now let's see if we can combine switches and relays so the circuit can "answer" slightly more interesting questions.

1.3. Switches, Relays, & Logic Gates

Say we want to build a simple dog washer circuit: a circuit that, based on some inputs, can tell us whether to wash our dog or not.

Our simple circuit is going to use a light bulb being on to mean yes, wash the dog. Light bulb off means no, don't wash the dog.

So we start with an extremely simple version with two switches.

In this first version, the switches are directly inside the bulb circuit. The person using the circuit can open or close each switch to answer a yes/no question.

Let's say switch 1 represents `STINKY`: whether the dog is stinky or not. Switch 2 represents `OLD_WASH`: has it been more than 5 days since the last wash.

So the rules for our first circuit are:

if `STINKY AND OLD_WASH`, the bulb is on.

Or in other words, if the dog is stinky and its last wash was over 5 days ago, then wash the dog.

Here is the circuit:

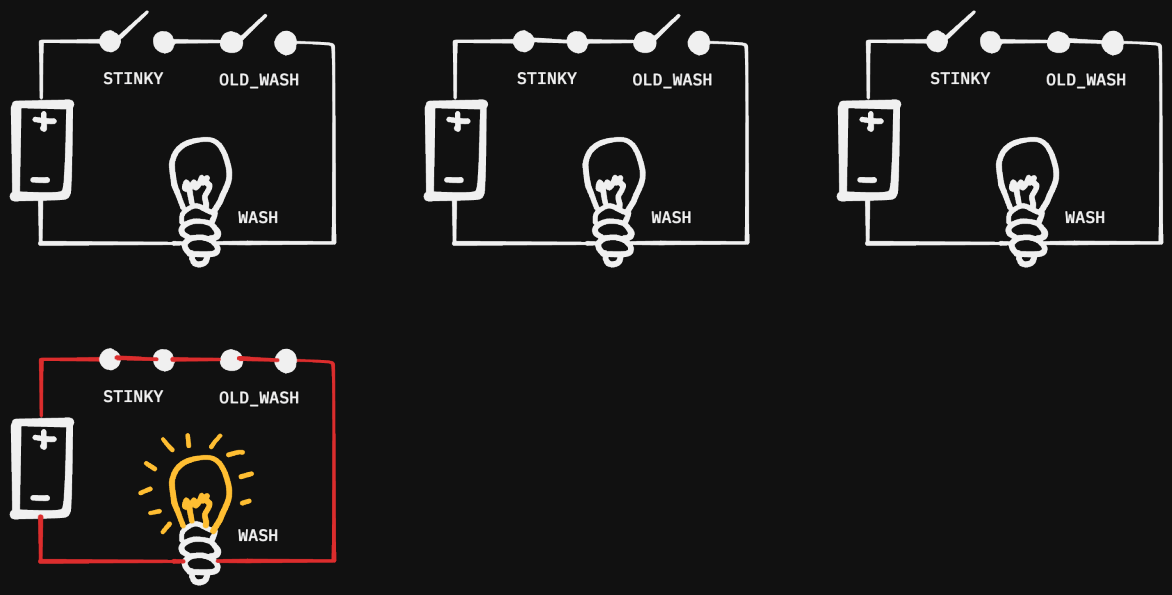


Diagram 4.1. The hand-switch version of AND.

This circuit shows a logical AND operation. A person is flipping the switches manually. The output turns on only when both inputs are true.

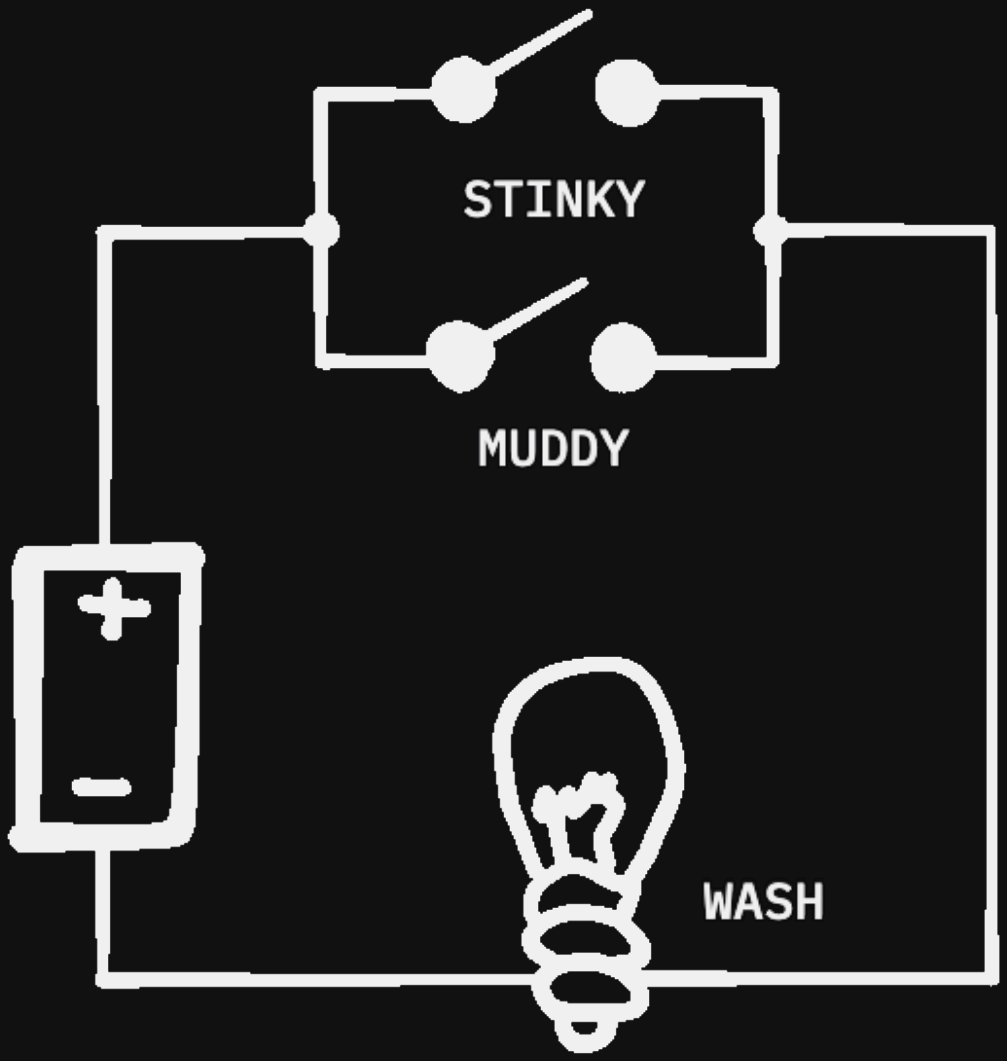
Now add a new input: `MUDDY`, if the dog is muddy.

Now the rules of the circuit change:

if `(MUDDY OR STINKY) AND OLD_WASH`

All this says is, if the dog is muddy or stinky and it's been more than 5 days since the dog's last wash, you should wash the dog.

Now let's focus on the (`MUDDY` OR `STINKY`) part of this circuit:



Full 6-frame animation at bytecolony.computer/writing/the-cpu/#f=or-gate-logical:1.

Diagram 4.2. The hand-switch version of OR.

This is a logical OR: either `MUDDY` or `STINKY` needs to be on for the bulb to turn on.

Before we combine these, let's tackle a bigger problem.

So far, every switch is flipped by a human. If we ever want to build a computer, it needs to run by itself: electricity has to be able to flip a switch by itself! But how?

Electromagnetic relays, that's how. Or at least, that is one early solution to this problem. We will talk about other solutions a little more later on.

This probably sounds quite complicated, but it is just a magnet powered by electricity.

One thing to mention before the next diagram: if you see several little batteries in a circuit, don't interpret that as several totally separate power sources. I am using the battery drawing as a symbol for "this point is connected to power," so the diagram doesn't turn into spaghetti.

Here is how it works:

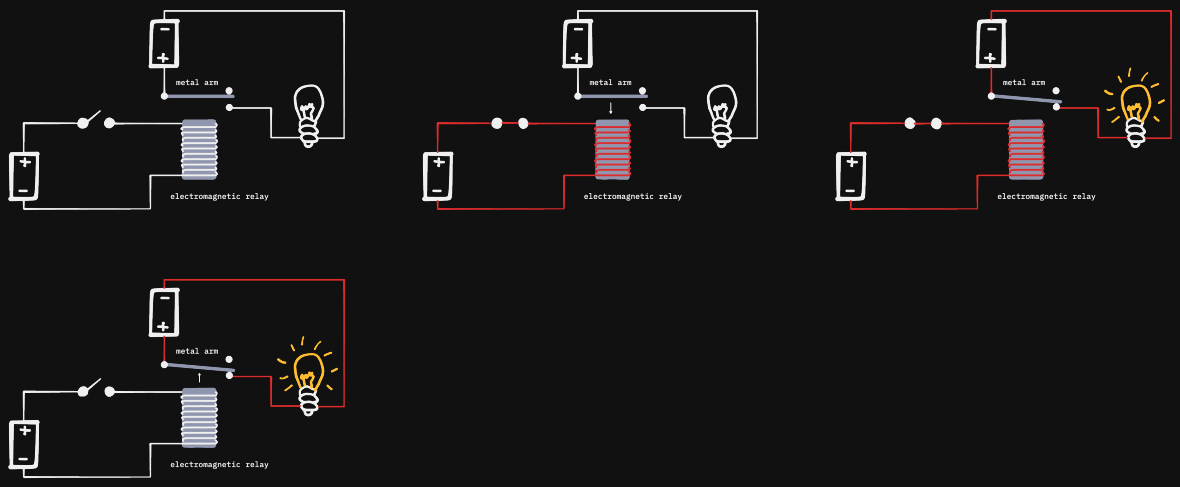


Diagram 4.3. An electromagnetic relay.

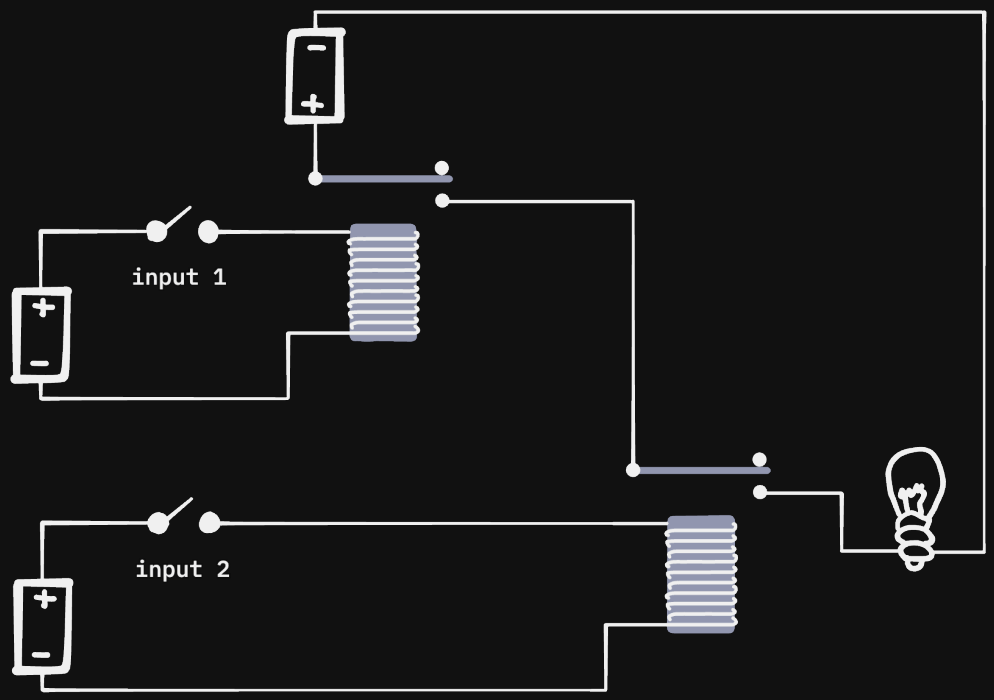
This relay is made from a coil of wire and a movable metal arm. When current flows through the coil, the coil becomes a magnet and pulls the arm down. When current stops, a spring pulls the arm back up.

A relay lets one circuit open or close a switch in another circuit. The two circuits stay electrically separate; the coil just magnetically pulls the arm, there is no electrical connection between them.

Also, in this example, we end up using a switch in the input circuit anyway, but any kind of electrical signal could be used, like the output of another circuit. The switch is just there to demonstrate how the relay works.

As you can also tell by the diagram, there is a slight delay between the coil turning on and the metal arm moving. Relays are mechanical, so they do not switch instantly.

Now let's see how we can build an actual electrical AND gate that takes two input wires and outputs an electrical signal.



Full 11-frame animation at bytecolony.computer/writing/the-cpu/#f=electronic-and-gate:1.

Diagram 4.4. An AND gate.

The output circuit has two breaks in it, one controlled by each input relay. Only when both inputs have signal do both relays close, completing the output loop.

Using these relays chained in clever ways, you can create every fundamental logic gate, such as the OR gate.

But before the next diagram, I am going to use one more new symbol: ground.

For our purposes here, the ground symbol will simply refer to the common return point of the circuit, usually connected to the negative side of the battery.

Every point marked with the ground symbol is connected together, as if there were hidden wires joining them underneath the drawing. It is not a new component. It is just a less messy way to draw the return path of the circuit.

The circuits are still loops. I am just not explicitly drawing the return wire anymore.

In a real schematic, the ground symbol itself would usually stay white. In these diagrams, I sometimes color it red when that return point is part of the active path for that frame. I think it makes the current path easier to follow visually.

This is how the ground symbol looks:

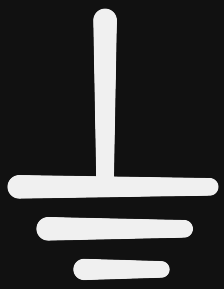
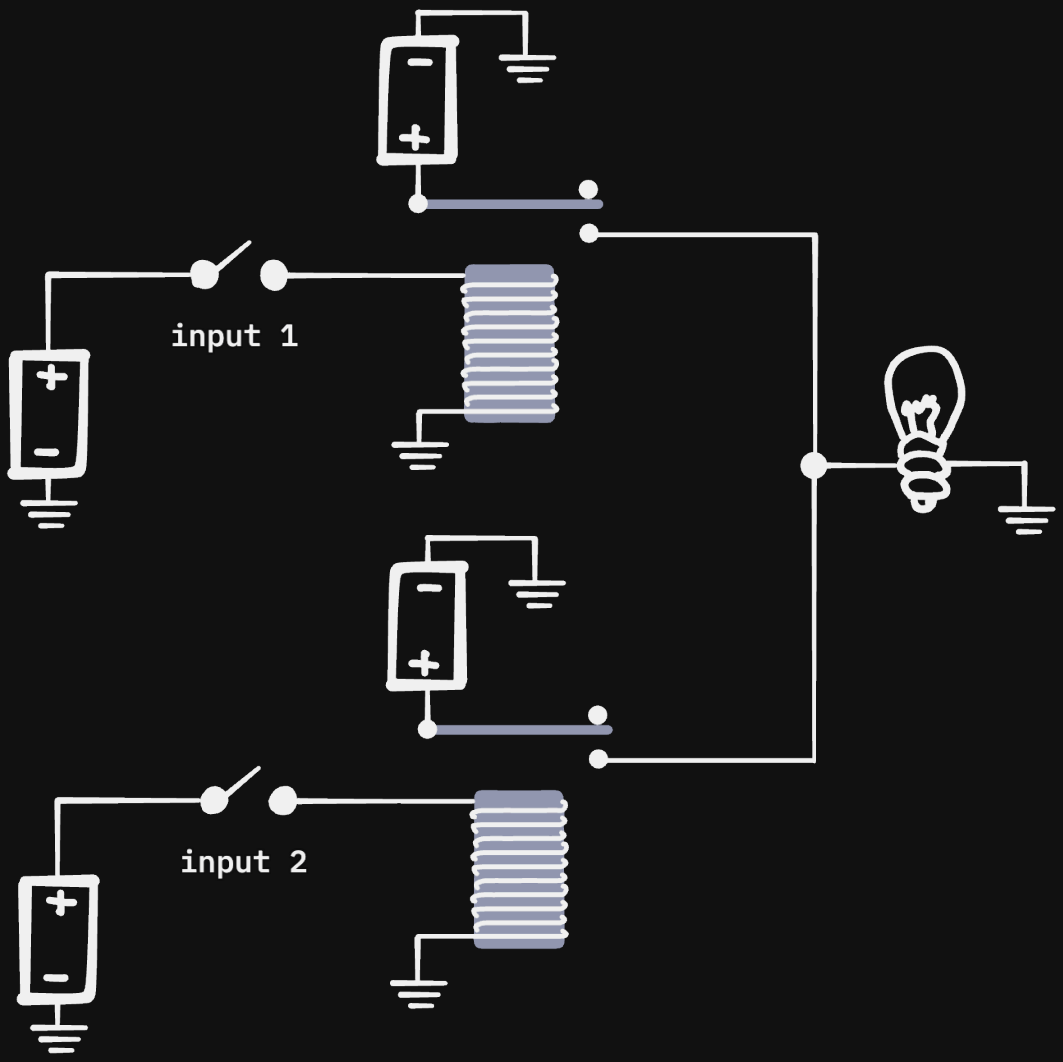


Diagram 4.5. The ground symbol.

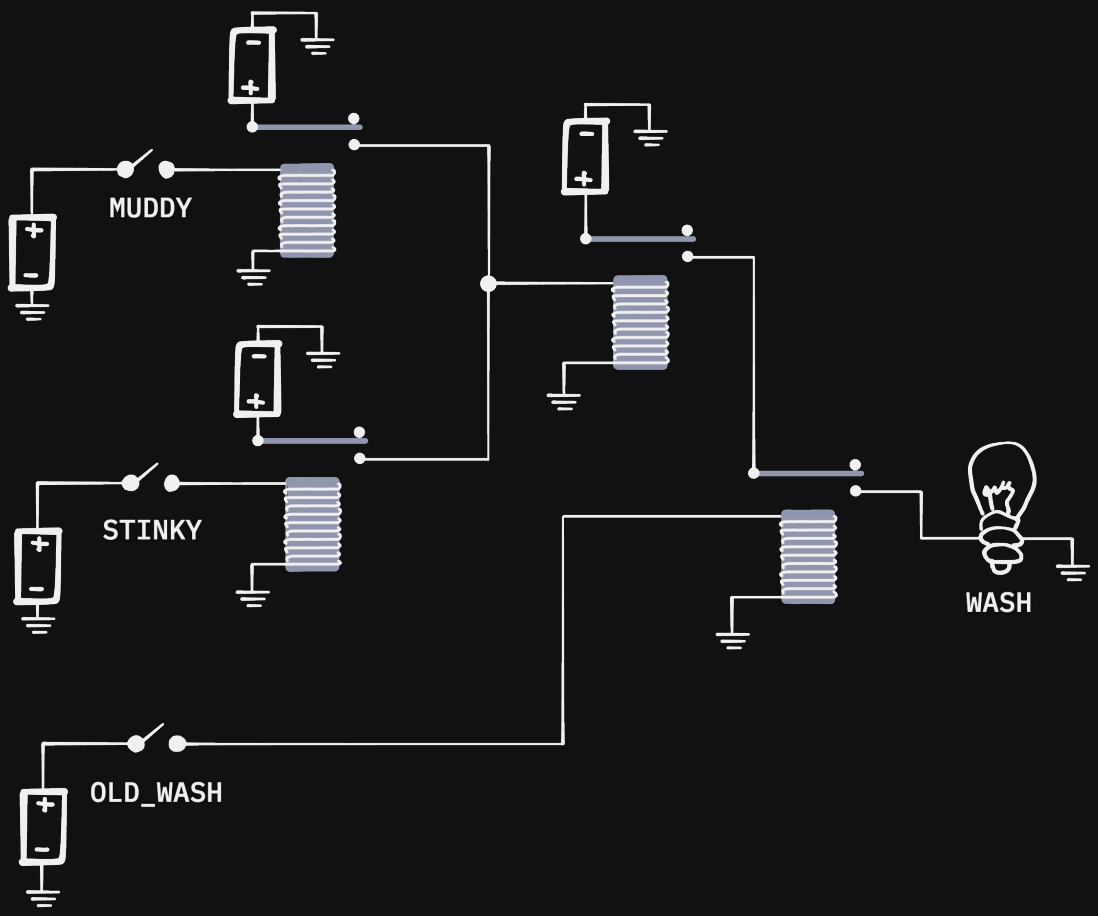
Now here is the OR gate:



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=electronic-or-gate:1.

Diagram 4.6. An electronic OR gate.

That is an OR gate using relays. Now here is the full dog washer circuit up to this point:



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=dog-washer-v1:1.

Diagram 4.7. The full dog washer circuit built with relays.

The animation does not show every possible combination of switches, only a handful. But in a nutshell, if `MUDDY` or `STINKY` is on, and `OLD_WASH` is also on, the bulb turns on.

Okay, now let's introduce one last input, or "sensor": `RAIN_SOON`, whether it is predicted to rain soon. The rules of the circuit change once again:

```
((MUDDY OR STINKY) AND OLD_WASH) AND NOT RAIN_SOON
```

The parentheses indicate order of operations. So in plain English:

If the dog is muddy or stinky and it's been more than 5 days since the dog's last wash and it's not going to rain soon, then wash the dog.

Let's focus on this NOT for a second. NOT just inverts a signal: if it receives signal, it outputs no signal; if it receives no signal, it outputs signal.

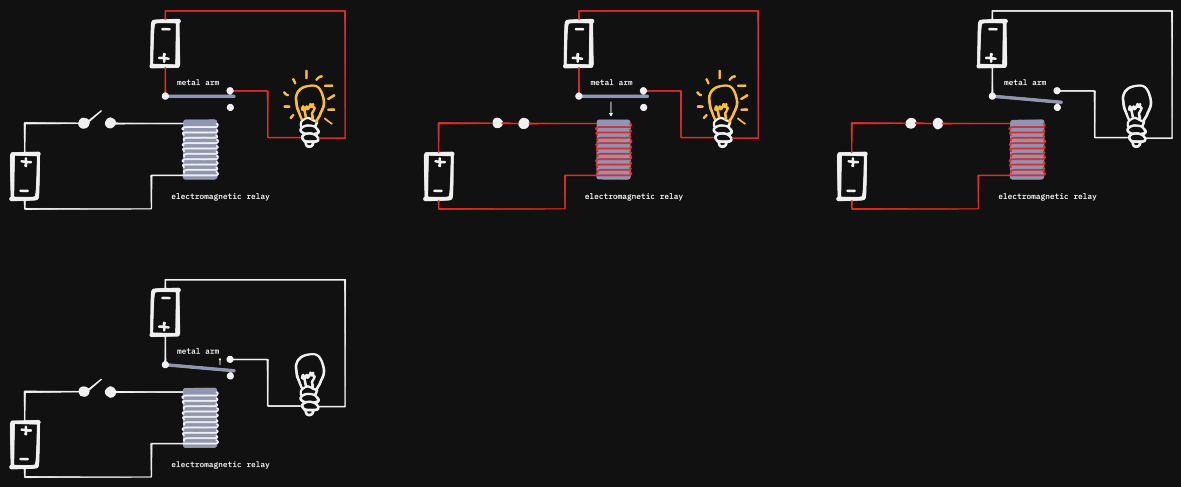


Diagram 4.8. A NOT gate.

Now let's clean up some of our understanding of circuits before we move on. We have been showing our outputs as a light bulb. For a bulb to be on, it needs to be connected to `+` and `-`, one on each side. That difference in voltage allows current to flow, turning on the bulb.

But a bulb is not always what we want. From now on, our gates' outputs will mostly feed other gates' inputs, so the output needs to be a wire, not a bulb. We can't just remove the bulb though; `+` connected directly to `-` would lead to a short-circuit. So what we do is either drive the wire up or down, so it is connected to either `+` or `-`. All of our relay gates can be simply adapted to do this.

A `1` output is a wire being driven high. A `0` output is not "nothing"; it is a wire being driven low. Remember this information; it will come in handy later on.

As you can see in the after example below, even when the relay is not pulling the arm, even when the output is 0, it is still touching the negative end of that battery, so it is being driven to `-`.

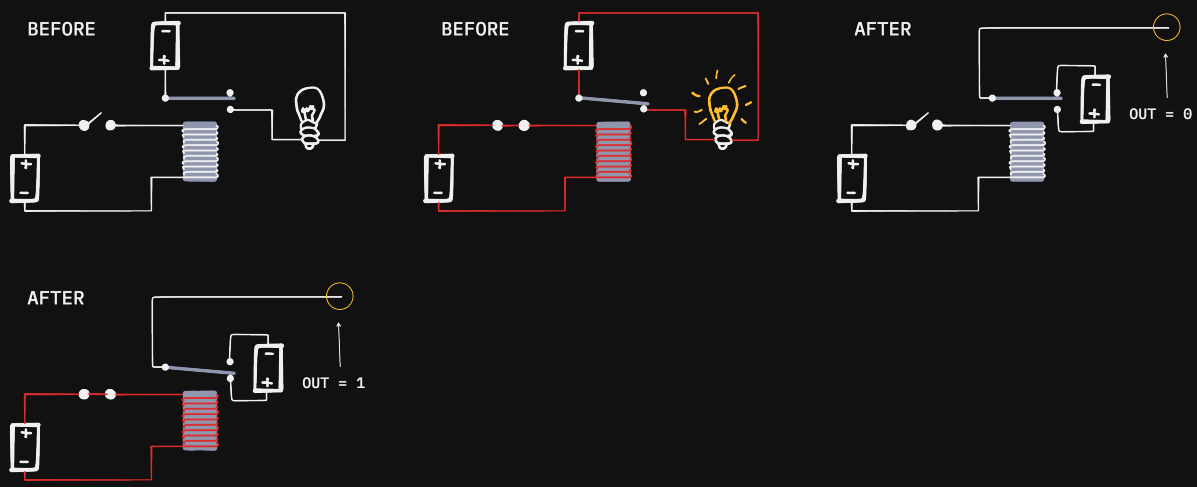


Diagram 4.9. Driving an output wire.

In this diagram, red wire means current is actively flowing. That's why `OUT = 1` is still white. Later when we stop drawing every logic gate, red wire will just mean high, or 1.

Now before we look at the completed circuit, let's learn some basic logic gate symbols.

An AND gate is drawn like this:

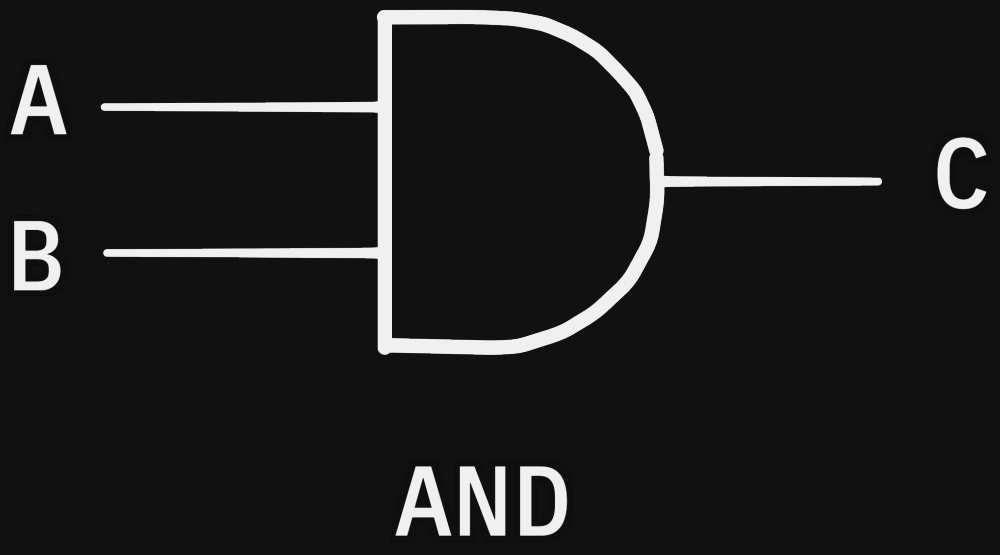


Diagram 4.10. An AND gate.

This symbol represents the AND circuit we made previously, except instead of turning a bulb on and off, it drives an output wire.

An OR gate is drawn like this:

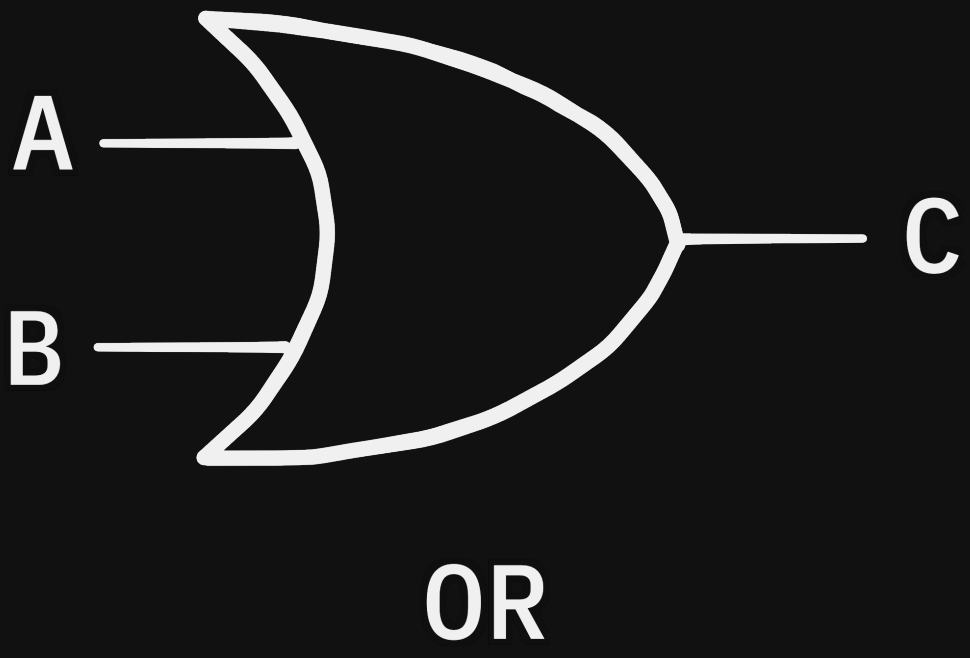


Diagram 4.11. An OR gate.

This symbol represents the OR circuit we made previously.

Whenever I use these symbols moving forward, they can almost directly translate to the circuits with the relays I showed you previously, but the internal components stay hidden for cleanliness.

Here are three more useful gate symbols:

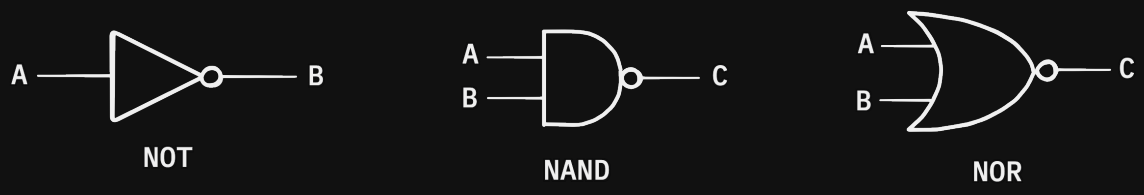
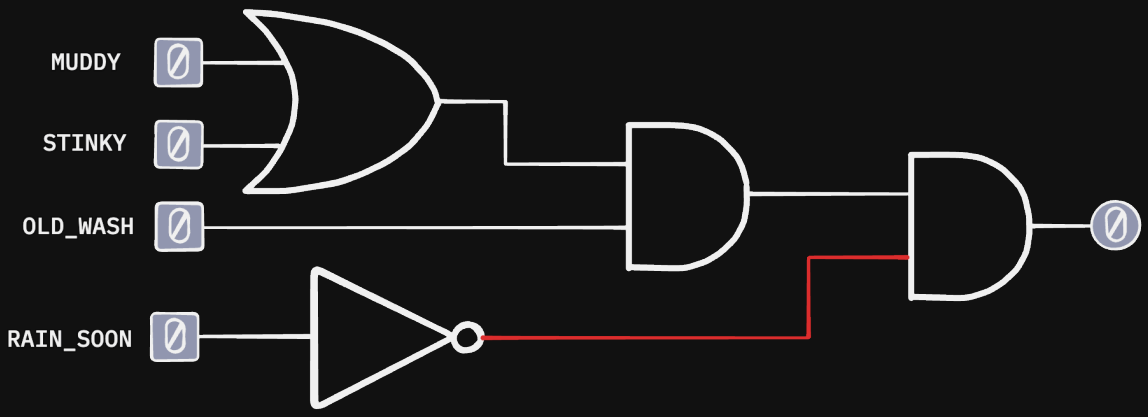


Diagram 4.12. NOT, NAND, NOR gates.

NAND is AND with the output flipped. NOR is OR with the output flipped.

That little circle at the end of a gate means "flip the output."

With our knowledge about logic gates, let's create the "should-I-wash-my-dog 5000" machine!



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=dog-washer-v2:1.

Diagram 4.13. The final dog washer circuit.

Again this animation doesn't cover all possible states.

Keep in mind these electromagnetic relays we used in the examples are quite big and slow.

Relays aren't the only solution. They are simply one of the early and intuitive methods to understand, and many real computers like the Harvard Mark I actually used these types of relays.

In modern computers, similar behavior is achieved by using transistors. If you want to learn more about transistor based logic gates: visit this site.

I don't know about you, but addition seems like a pretty logical next step to these logic gates. But not so fast.

This is how circuits make yes/no decisions. Not by understanding what `MUDDY` means, but by wiring simple gates so the output turns on only for the input pattern we care about.

A wire is just a wire. We gave these wires meaning. We decided that one wire means `STINKY`, another wire means `MUDDY`, and another means `RAIN_SOON`.

To make a CPU, we need to give wires a different kind of meaning: numbers. Before we can build a circuit that adds, we need a way to represent numbers using only on and off.

That is what the next section is about.

1.4. Counting With Wires

Before we continue with this section, let's define some terms.

A wire driven low is `0`, and a wire driven high is `1`. Let's call one wire one bit. A bit can either be `0` or `1`.

These are just labels that represent the state of a wire.

A group of 8 bits is called a byte. With 8 bits, there are `2^8`, or 256, possible patterns. So if we use those patterns to represent non-negative numbers, one byte can represent 0 through 255.

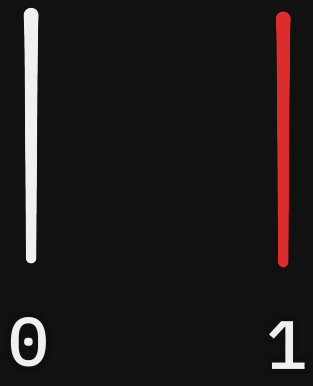
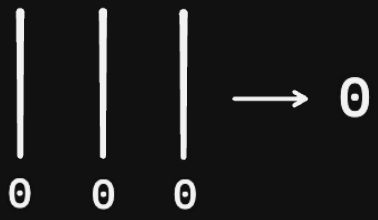


Diagram 5.1. One wire can represent two states: `0` or `1`.

If we want to represent numbers using wires, we are going to need more than one wire, because one wire can only represent up to two numbers, since it only has two possible states: `0` or `1`.

But two wires have `2^2`, or four states, and three wires have `2^3`, or eight states. That would allow us to represent more numbers.

Here are all the possible states we have with 3 wires:



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=3-states:1.

Diagram 5.2. States with 3 wires.

We can represent 8 numbers just like this.

But, why does `010` mean 2? Why does `101` mean 5? Is it just randomly assigned?

Not exactly. To understand this, let's take a quick detour to decimal, a.k.a. base ten.

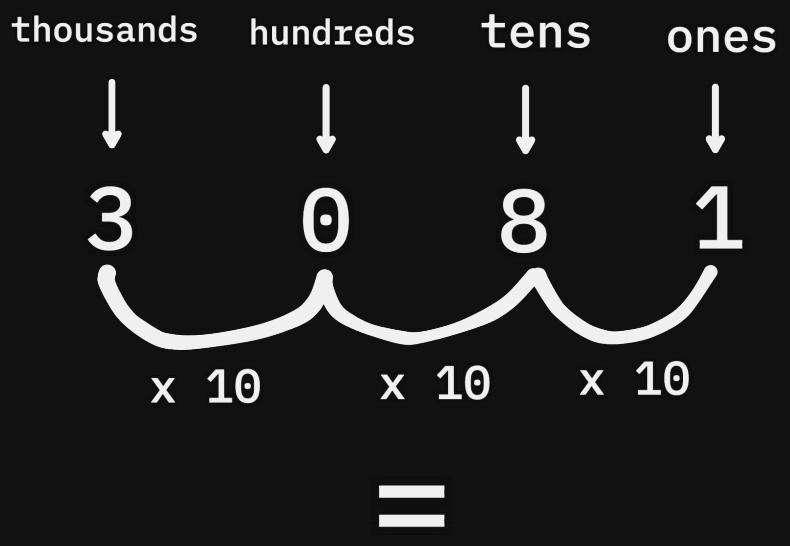


Diagram 5.3. The decimal system.

In our decimal counting system, each place value is a power of 10. That is because we have ten digits: 0-9.

This exact same place value logic can apply to the binary system too. We have two digits, 0 and 1, so each place is a power of 2.

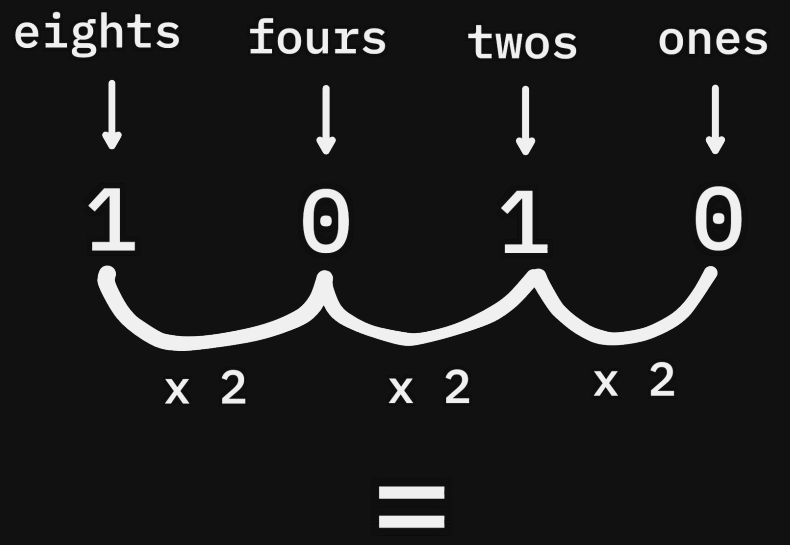


Diagram 5.4. The binary system.

So binary is, at the end of the day, decimal but with only two digits instead of ten.

A few examples:

- `101` means 5
- `1101` means 13
- `101010` means 42
- `1100011` means 99

You don't need to do these problems in your head, but I hope the idea of how binary works makes sense.

Let's walk through `1101` together.

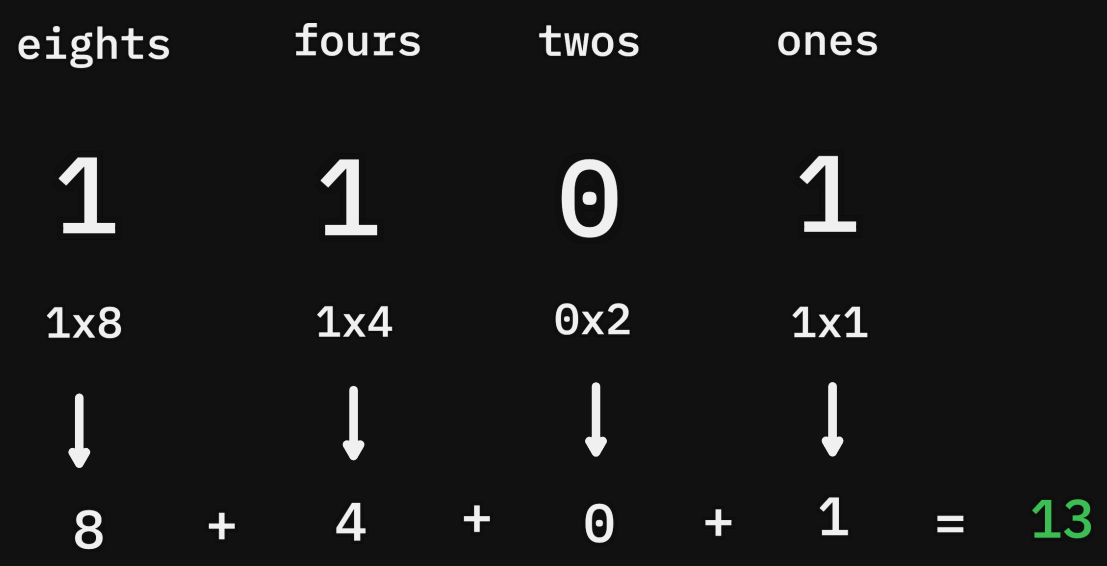


Diagram 5.5. An example in binary.

So now that we can represent numbers with wires, how can we add numbers together? That is what the next section is all about.

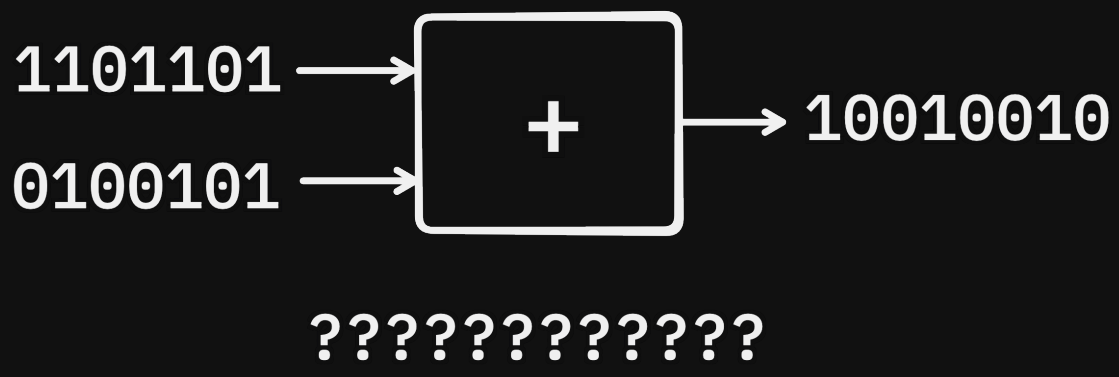


Diagram 5.6. Addition?

1.5. Addition

Let's start with a brief reminder of how we algorithmically add two decimal numbers.

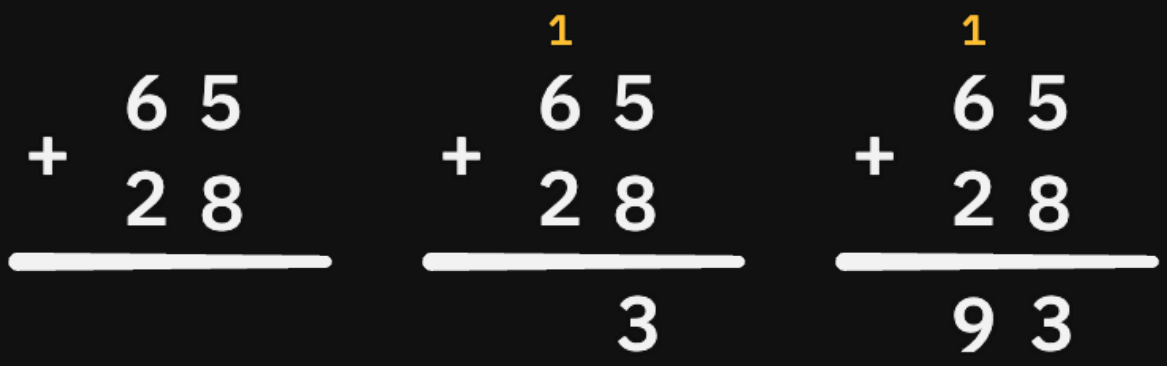
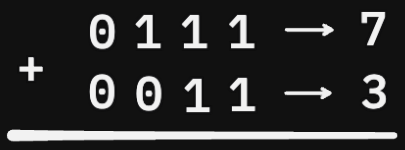


Diagram 6.1. Standard decimal addition.

We start at the rightmost column, do 5+8, get 13, and carry the 1. So we write 3 as the sum, and 1 as the carry. We then move left and repeat over and over remembering to add any carry-in values. Binary addition works the same way.



Full 6-frame animation at bytecolony.computer/writing/the-cpu/#f=binary-addition:1.

Diagram 6.2. Binary addition.

`1 + 1` gives `10`, which is binary for 2.

So the sum bit for that column is `0`, and the carry is `1`.

`1 + 1 + 1` gives `11`, which is binary for 3. So the sum bit is `1`, and the carry is `1`.

How do we build a circuit using logic gates that performs this standard addition algorithm?

Well, let's start with the rightmost column. If we think about it, all the possible states are:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

So just `0 + 0`, `1 + 0`, `1 + 1`, or `0 + 1`. If we can make a tiny circuit that takes two inputs, and produces two outputs that match these combinations, we have added the first column.

This is called a half adder. A half adder adds two bits, but it does not handle a carry-in value. That is the job of a full adder.

Let's first build this half adder.

Let's start by computing the sum, not the carry-out.

This is what we want our circuit to do:

A	B	Sum
0	0	0
0	1	1
1	0	1
1	1	0

The sum is `1` only when exactly one input is `1`.

This is called XOR, short for exclusive OR.

If we combine an OR gate and a NAND gate, and AND them together, we get XOR:

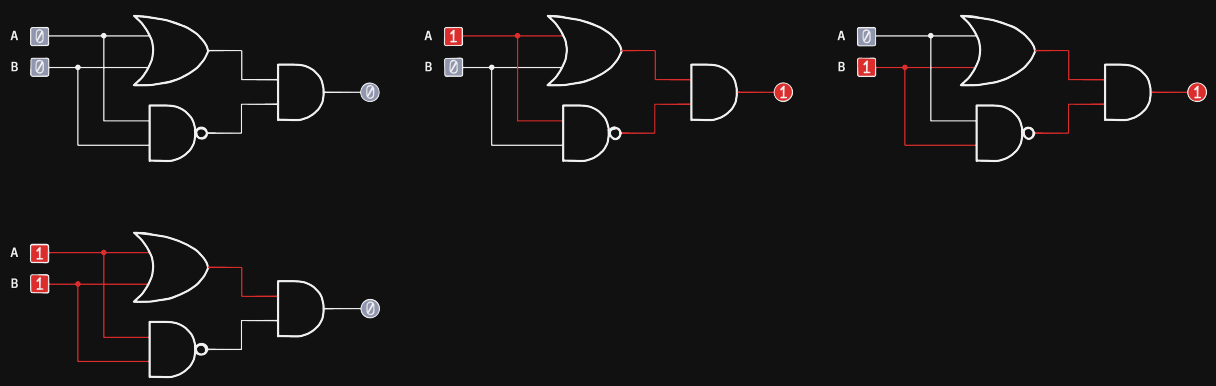


Diagram 6.3. Half adder sum / XOR.

OR checks that at least one input is on, and NAND makes sure that both inputs are not on.

Here is how an XOR gate looks:

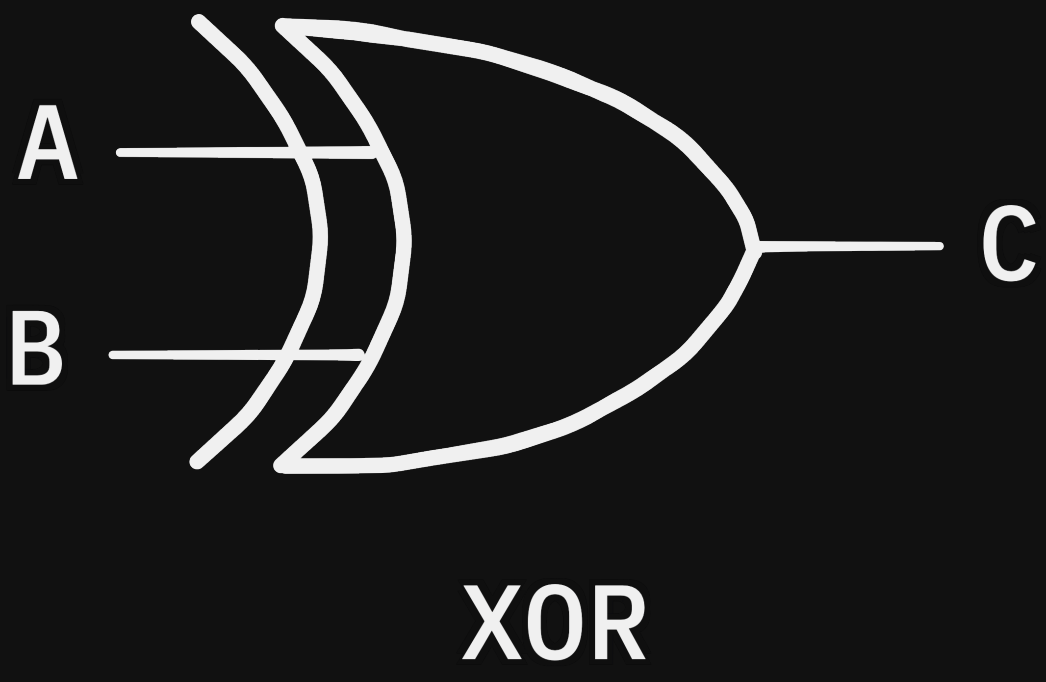
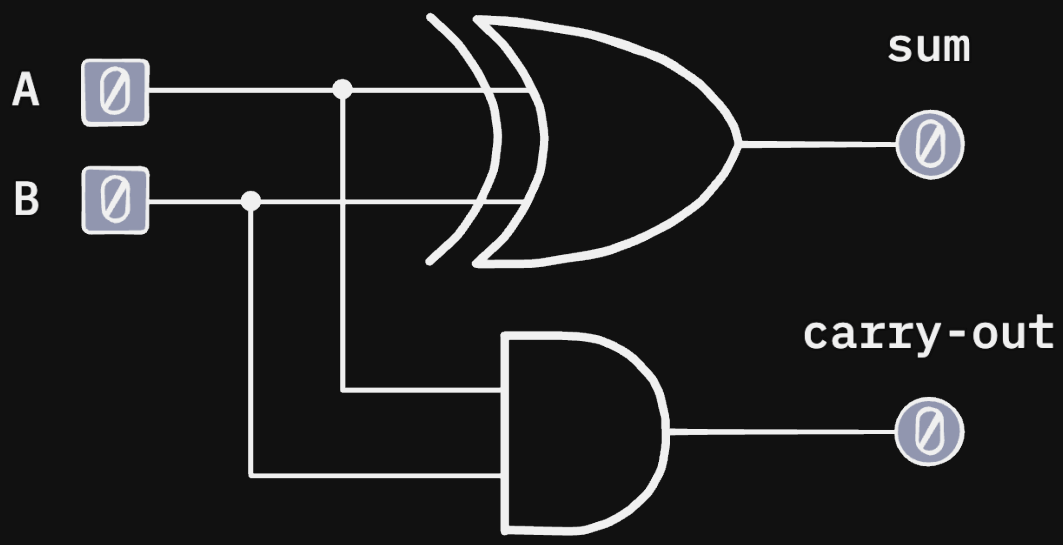


Diagram 6.4. An XOR gate.

Now let's do the carry value. The carry is simple! We only want to carry if we are doing `1 + 1`, so we just use an AND gate to check if both inputs are on.

Now here is our half adder:



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=half-adder:1.

Diagram 6.5. A half adder.

As you can see, it works! `0 + 0 = 0`, `1 + 0 = 1`, `0 + 1 = 1`, and `1 + 1 = 10`.

Now let's package up our half adder into a little box. From now on, I will call these packaged-up circuits chips:



Diagram 6.6. A half adder chip.

Now that we have a half adder, we can add the rightmost column. That works because the rightmost column has no carry-in from a previous column. It only needs to add two bits.

So if we have a number like this:

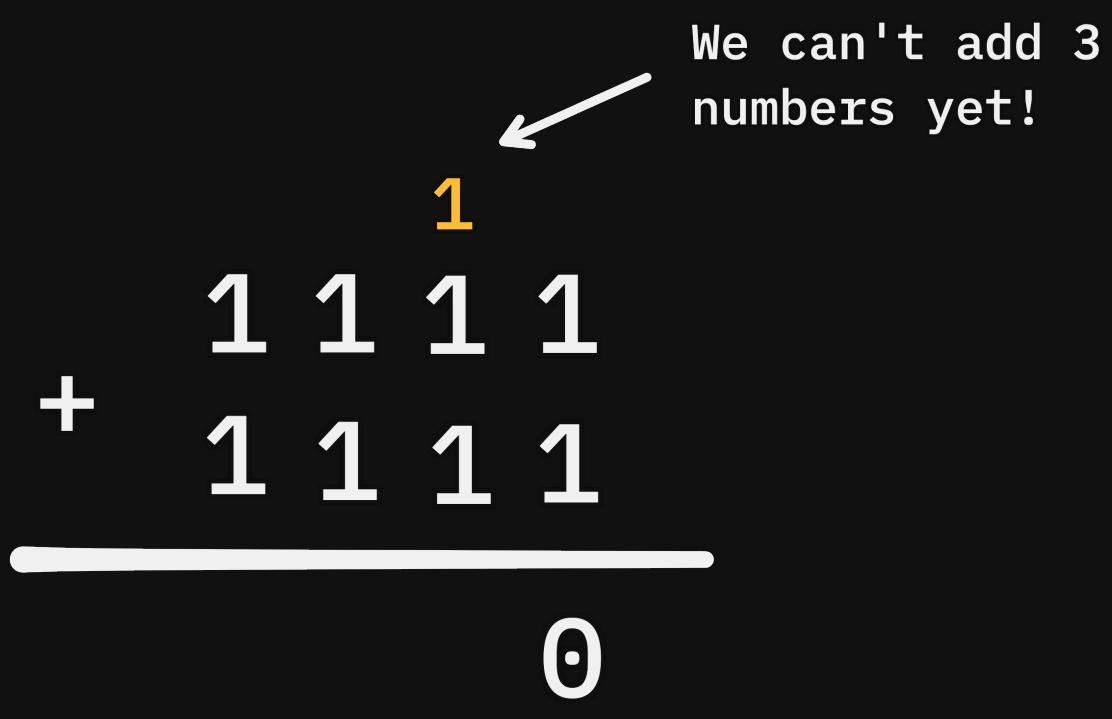


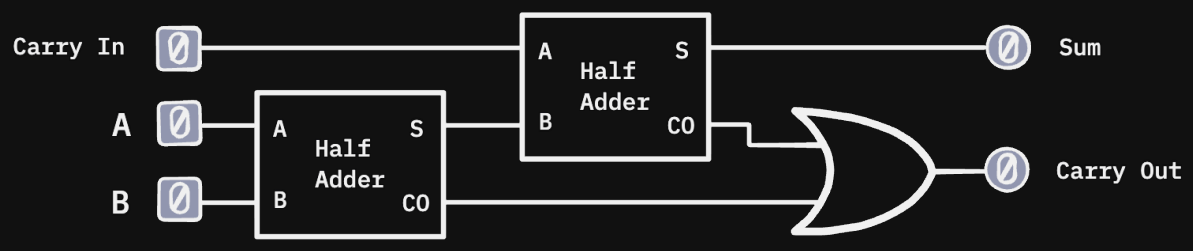
Diagram 6.7. The next column has to add two bits plus a carry-in.

The half adder can handle the first column: `1 + 1`. That gives us a sum bit of `0` and a carry-out of `1`.

But now the next column has three things to add: `1 + 1 + 1`. The two original bits, plus the carry from the previous column.

A half adder cannot do that. It only accepts two inputs. To continue adding up the other columns, we need a circuit that can take in three inputs: `A`, `B`, and `carry-in`.

To add three bits, we use two half adders and an OR gate:



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=full-adder:1.

Diagram 6.8. A full adder.

This might look confusing at first. What if both half adders output a carry-out at the same time?

That actually never happens. If a half adder outputs a carry, the sum bit is always 0. So both can never output carries at the same time. Take a moment to think about this if you are confused.

So we can confidently OR the two carry outputs together. If either one is `1`, the full adder's carry-out is `1`.

Let's again package this up into a chip:

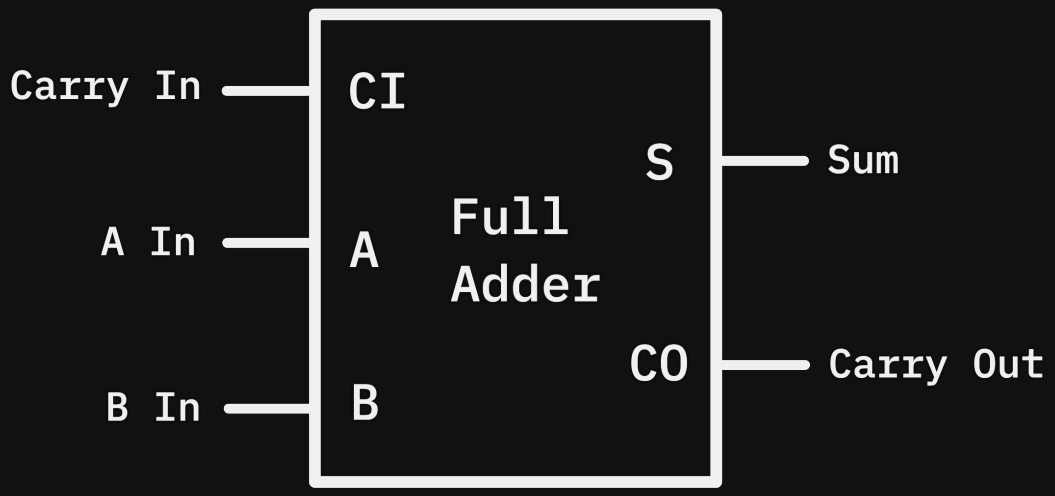


Diagram 6.9. A full adder chip.

We have made a full adder!

Now we can chain full adders together to add two 8-bit numbers. Since 8 bits make one byte, this is an adder that can add two one-byte numbers: anything from 0 to 255.

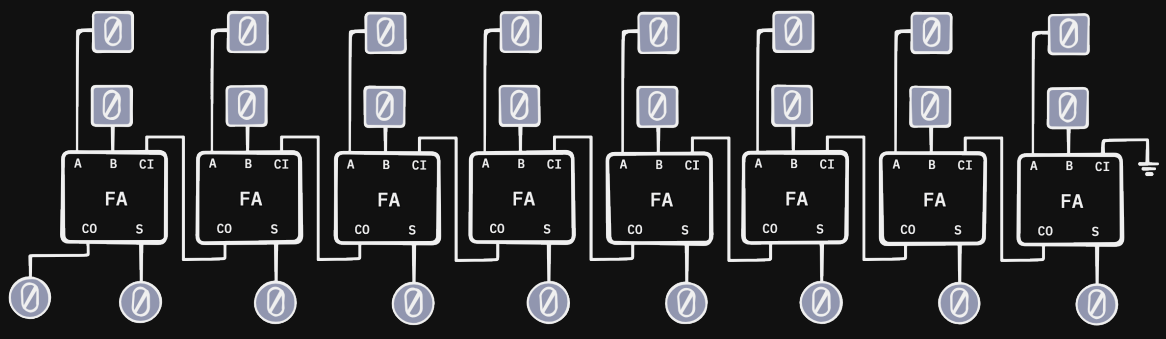


Diagram 6.10. An 8-bit adder.

Each full adder handles one column. The carry-out from one column becomes the carry-in for the next column. That is it! That is all addition is!

Keep in mind, carry-in for the first adder is set to ground, a.k.a. 0.

Also, notice how we have 9 outputs, not 8. That is because two 8-bit values can add up to a number too big to fit in eight bits. It's like how adding two 2-digit numbers could result in a three-digit number for us. Like `50+50=100`.

Now let's package this up into a chip once again:

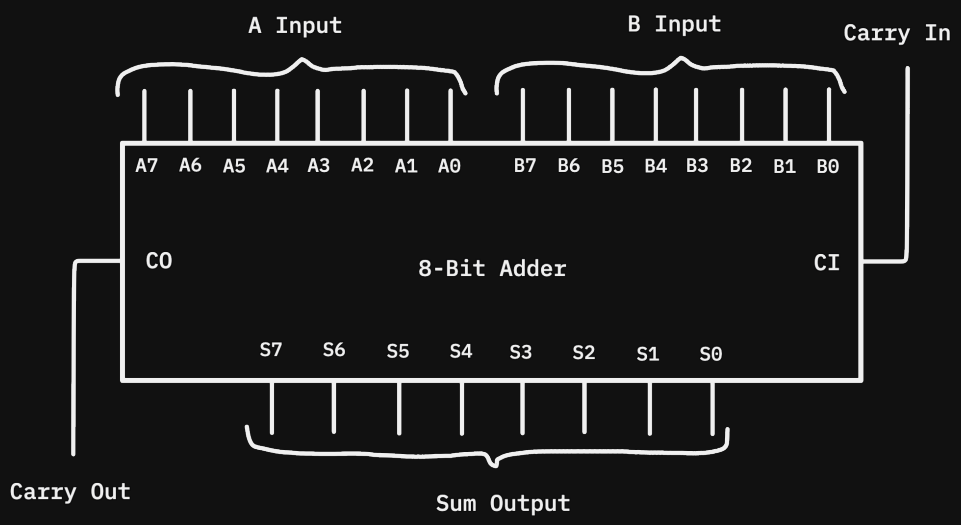


Diagram 6.11. An 8-bit adder chip.

Now we have the carry-out and carry-in as separate inputs and outputs and the whole adder nicely organized into this chip.

Let's have a look at some example problems:

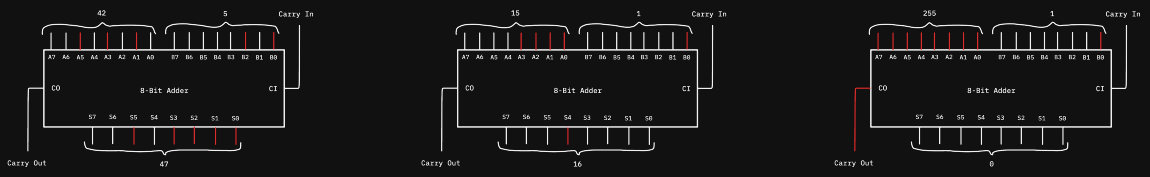


Diagram 6.12. Some examples on the adder.

As you can see in the third example, adding 1 to 255 turns every sum bit to `0` and turns the carry-out on.

This doesn't mean the adder got the wrong answer. In fact, `255 + 1` is `1 00000000` in binary: eight `0` output bits, plus one extra carry-out bit on the left. If we only look at the one-byte output, the result looks like `00000000`, or 0. If we also look at the carry-out, we can see that the real answer was 256.

That is called an overflow: the result was too large to fit inside one byte, so the extra information spilled out into the carry-out bit.

The adder can also produce little status wires, called flags.

For example, if the answer is `00000000`, a `ZERO` flag can turn on. If addition spills past one byte, a `CARRY` flag can turn on. So `11111111 + 00000001` gives `00000000` with carry-out `1`.

I don't want to go deep into flags yet. Just remember that the adder can output little yes/no facts about the sum. That matters later for instructions like "jump if zero." But let's not get ahead of ourselves.

We have just built addition! But we also need something else: storage.

For example, let's say we want to build a circuit that counts by ones, like 1, 2, 3, 4,…

The obvious idea is to feed the output of the adder back into one of its inputs. Start with `00000000`, add `00000001`, get `00000001`. Feed that back in, add `00000001` again, get `00000010`. Then `00000011`, then `00000100`, and so on.

That seems correct at first glance.

But there is a big problem. An adder just looks at its current inputs and computes an output.

So if we wire the output straight back into the input, there is no stable value anymore. The adder is basically being asked to make a number equal to itself plus one:

```
input = input + 1
```

That can never settle. As soon as the output changes, the input changes too, which means the output has to change again, which means the input changes again.

With relays, you might physically see this mess play out. With transistors, it would happen almost instantly.

There is no boundary between the old value and the new value.

There is no clean "step 1, step 2, step 3."

So this is not enough. We need a circuit that can hold a value still, then update it only when we tell it to.

That is the next problem: memory.